VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

OBJECT ORIENTED PROGRAMMING
(25MC409)
I M.C.A II Semester
2025-26 EVEN SEMESTER

Name of the Faculty : Dr. K. Gayatri
Assistant Professor, Department of CA
VFSTR (Deemed to be University)

Institute Vision & Mission

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

VISION:

To evolve in to a centre of excellence in Science & Technology through creative and innovative practices in teaching-learning, towards promoting academic achievement and research excellence to produce internationally accepted, competitive and world class professionals who are psychologically strong & emotionally balanced imbued with social consciousness & ethical values.

MISSION:

To Provide High Quality academic programmes, training activities, research facilities and opportunities supported by continuous industry - institute interaction aimed at promoting employability, entrepreneurship, leadership and research aptitude among students and contribute to the economic and technological development of the region, state and nation.

Department Vision & Mission

VISION:

To emerge as a renowned center in the field of computer applications for providing high quality education and research to produce globally competent professionals with ethical values.

MISSION:

- **M1:** To provide high quality education with cutting-edge curriculum and learner-centered approach.
- **M2:** To establish the culture of innovation and research through industry-academia linkages.
- **M3:** To cultivate ethical and socially-conscious professionals with leadership qualities, who understand the global implications of technology.

PO' s of the Department

- **PO1 (Foundation Knowledge):** Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.
- **PO2 (Problem Analysis):** Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.
- **PO3 (Development of Solutions):** Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- **PO4 (Modern Tool Usage**): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO' s of the Department

- **PO5 (Individual and Teamwork):** Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- **PO6 (Project Management and Finance)**: Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.
- **PO7 (Ethics):** Commit to professional ethics in managing software projects with financial | aspects. Learn to use new technologies for cyber security and insulate customers from malware
- **PO8 (Life-long learning):** Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

PEO' s of the Department

- **PEO1:** Develop cutting-edge technical skills to succeed in career and as an entrepreneur.
- **PEO2 :** Pursue higher education and research with critical thinking and problem solving capabilities
- **PEO3:** Demonstrate ethical, collaborative, and leadership abilities in multi-disciplinary domains.

PSO's of the Department

- PSO1: To provide solutions for complex computational problems with requisite mathematical, programming languages and tools.
- PSO2: To design, develop and analyse software and related services using algorithms, web design, and Big Data Analytics for real time applications.

Module-2

Unit-3

AWT, APPLETS AND GUI PROGRAMMING WITH SWING: Applet life cycle, AWT, AWT Hierarchy, AWT Controls, Delegation Event Model.

EXPLORINGSWING CONTROLS: JLabel, JTextField, JButton, JCheckBox, JRadioButton, JTabbed Pane, JList, JCombo Box.

GUI Programming

An Application is a program or group of programs designed for end users.

In Java, we have 2 types of Applications

- CUI Applications
- GUI Applications

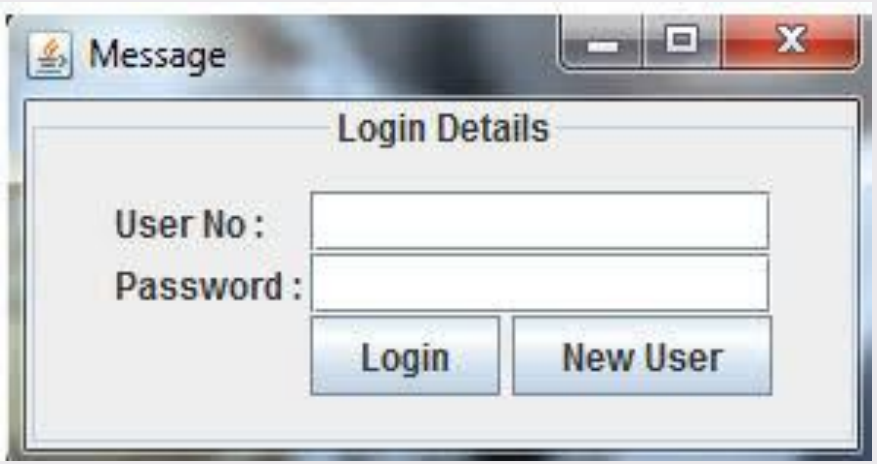
1.CUI Applications [Character User Interface]

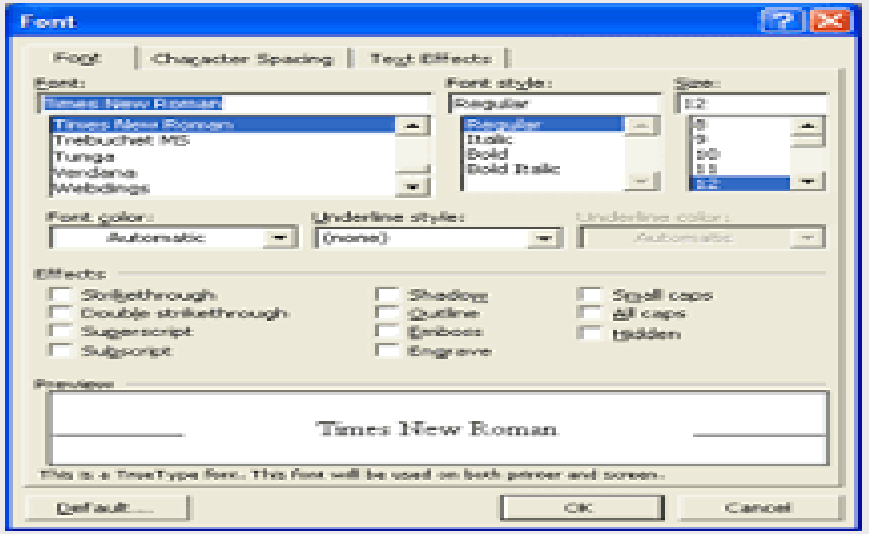
Character user interface or command-line user interface, CUI is a way for users to interact with computer programs. It works by allowing the user (client) to issue commands as one or more lines of text (referred to as command lines) to a program.

GUI

- GUI stands for Graphical User Interface.
- It refers to an interface that allows one to interact with electronic devices like computers and tablets through graphic elements.
- It uses icons, menus and other graphical representations to display information, as opposed to text-based commands.
- The graphic elements enable users to give commands to the computer and select functions by using mouse or other input devices.

Examples of GUI Applications





GUI in JAVA Programming

Abstract Window Toolkit (AWT) is an API to develop GUI or window – based applications in java.

A graphical user interface is built of graphical elements called components.

Typical components include such items as buttons, scrollbars, and text fields.

In java GUI can be design using some predefined classes. All these classes are defined in java.awt package.

Design of GUI in JAVA

There are two types GUI designs in JAVA

1. Applet

- Applet is based on the **Applet class**.
- These applets use the Abstract Window Toolkit (AWT) to provide the graphical user interface .
- This style of applet has been available since Java was first created.

2. JApplet

- The second type of applets are those based on the Swing class **JApplet**, which inherits **Applet**.
- Swing applets use the Swing classes to provide the GUI.
- Swing offers a richer and often easier-to-use user interface than does the AWT

Advantages of GUI over CUI

- GUI provides graphical icons to interact while the CUI (Character User Interface) offers the simple text-based interfaces.
- GUI makes the application more entertaining and interesting on the other hand CUI does not.
- GUI offers click and execute environment while in CUI every time we have to enter the command for a task.
- New user can easily interact with graphical user interface by the visual indicators but it is difficult in Character user interface.

Advantages of GUI over CUI

- GUI offers a lot of controls of file system and the operating system while in CUI you have to use commands which is difficult to remember.
- Windows concept in GUI allow the user to view, manipulate and control the multiple applications at once while in CUI user can control one task at a time.
- GUI provides multitasking environment so as the CUI also does but CUI does not provide same ease as the GUI do.
- Using GUI it is easier to control and navigate the operating system which becomes very slow in command user interface. GUI can be easily customized.

Java AWT

- Java AWT (Abstract Window Toolkit) is *an API to develop GUI or window-based applications* in java.

- The java.awt package provides classes for AWT API such as

  - ✓ TextField
  - ✓ Label
  - ✓ TextArea
  - ✓ RadioButton
  - ✓ CheckBox
  - ✓ Choice
  - ✓ List

AWT - Controls

Every user interface considers the following three main aspects:

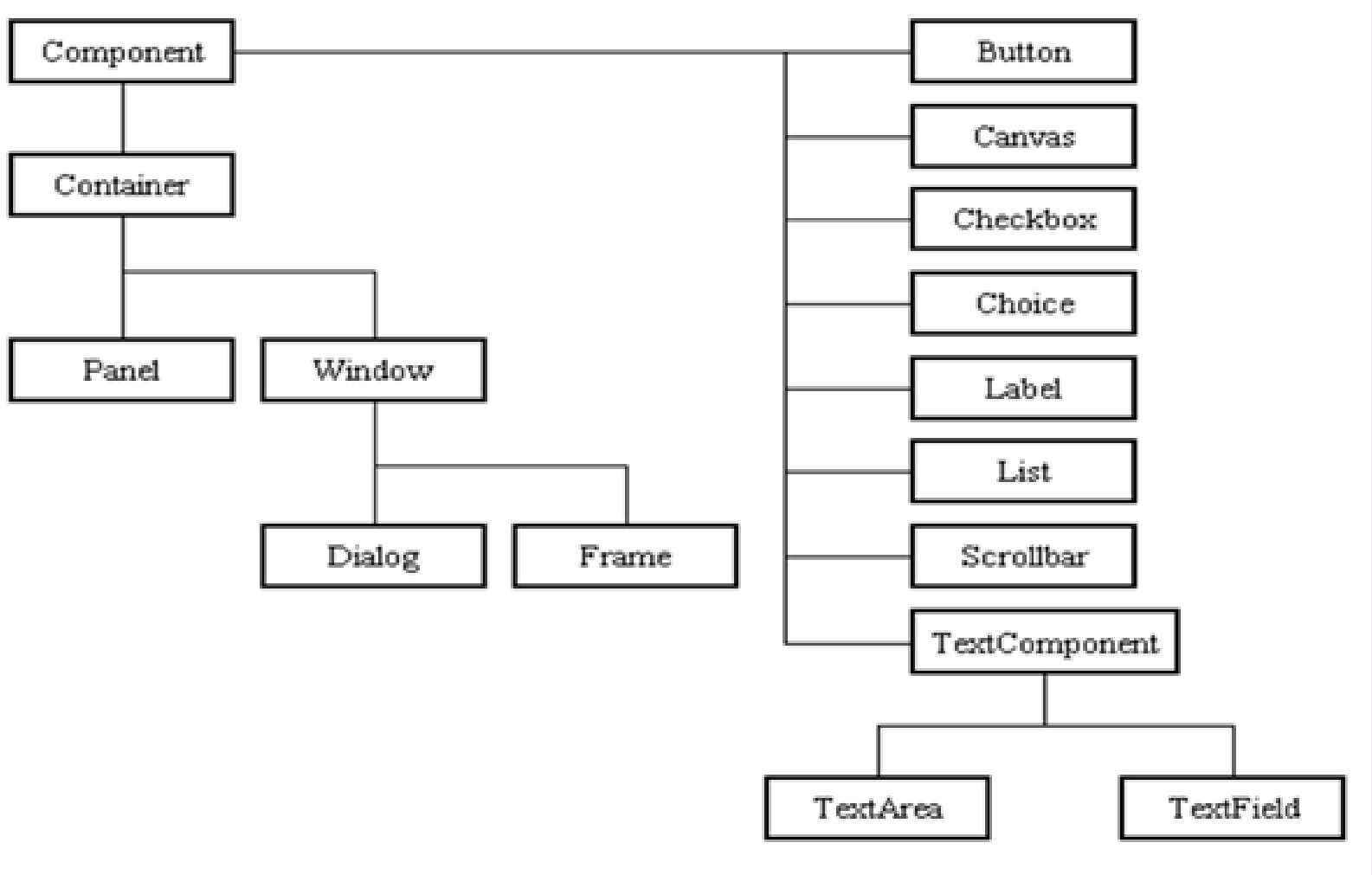
- UI elements : These are the core visual elements the user eventually sees and interacts with. GWT provides a huge list of widely used and common elements varying from basic to complex which we will cover in this tutorial.

- Layouts: They define how UI elements should be organized on the screen and provide a final look and feel to the GUI (Graphical User Interface). This part will be covered in Layout chapter.

- Behavior: These are events which occur when the user interacts with UI elements. This part will be covered in Event Handling.

AWT Hierarchy

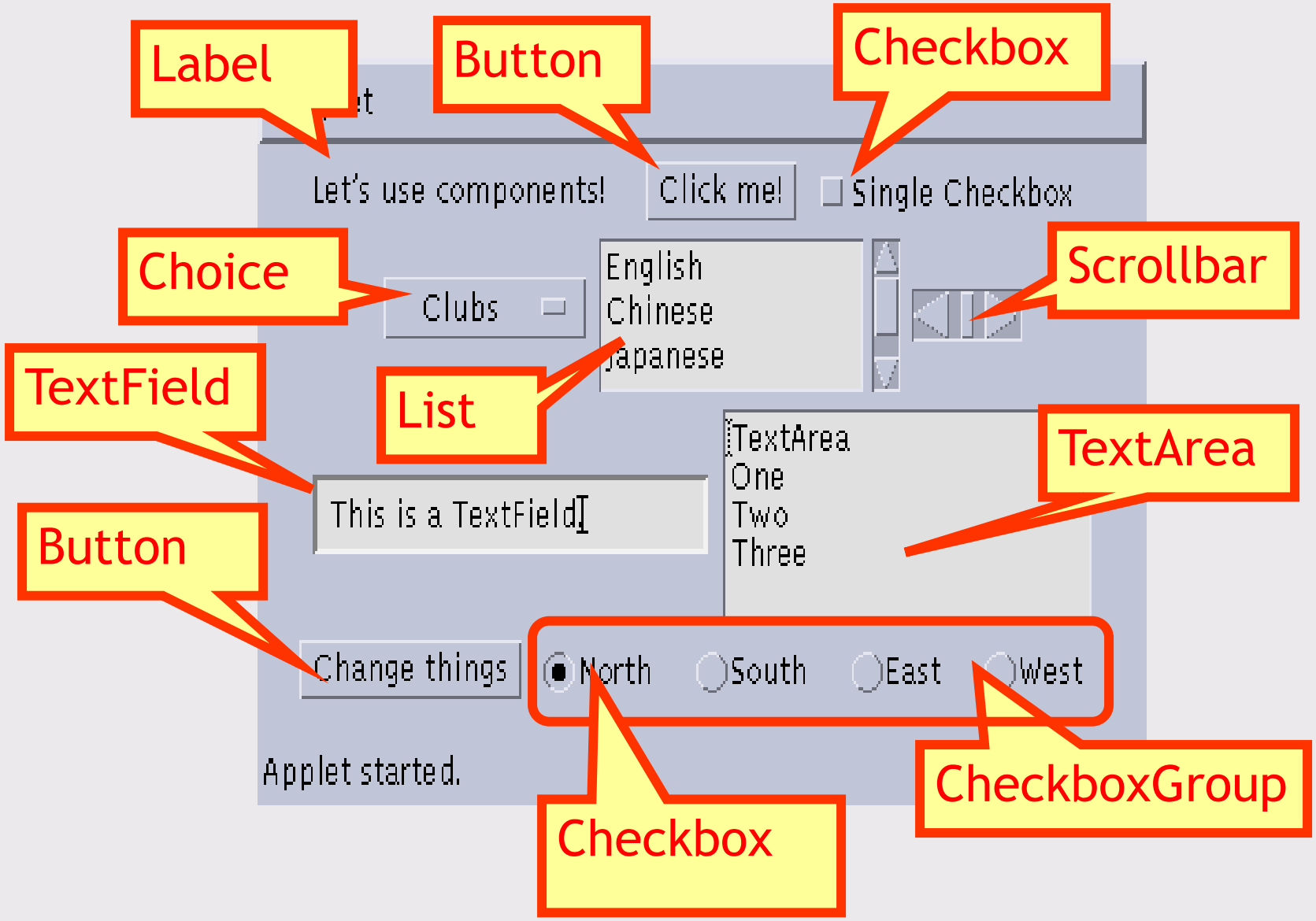
AWT CLASS HEIRARCHY



Component

- ✓ Component class is at the top most class of the AWT hierarchy.
- ✓ A component is an object having graphical representation that can be displayed on the screen and interact with the user. Examples of components are buttons ,checkboxes etc.
- ✓ All user interface elements that are displayed on the screen and that interact with the user are subclasses of Component.
- ✓ Anything that is derived from the component class is can be a component.
- ✓ The class contains no of methods for event handling, manging graphics painting and sizing and resizing the window.

Some types of components



Container

- ✓ Container class is a subclass of Component.
- ✓ It has additional methods that allow other Component objects to be nested within it.
- ✓ A container object is component that can contain other AWT components.
- ✓ Some of the containers are Frames, Dialogs, windows, panel (applet)
- ✓ Responsibility of a container : To lay - out (that is, positioning) any components that it contains.

Frame

- ✓ It is a subclass of Window and has a title bar, menu bar, borders, and resizing corners.
- ✓ Two forms of Frame constructors :
  - ❑ Frame() : It creates a standard window that does not contain a title.
    Frame f=new Frame();
  - ❑ Frame(String *title*) : It creates a window with the title specified by *title*.
    Frame f=new Frame("my frame");

Panel

- ✓ The Panel class is a concrete subclass of Container.
- ✓ It doesn't add any new methods; it simply implements Container.
- ✓ A panel provides space in which an application can attach any other component, including other panels
- ✓ It is a concrete screen component. Applet class is the sub class of panel
- ✓ Applet's output is drawn on the surface of a Panel object. In essence, a Panel is a window that does not contain a title bar, menu bar, or border.

- ✓ This is why we don't see these items when an applet is run inside a browser.
- ✓ When we run an applet using an appletviewer, the applet viewer provides the title and border.
- ✓ Other components can be added to a Panel object by its add() method (inherited from Container).
- ✓ Once these components have been added, we can position and resize them manually using the setLocation(), setSize(), or setBounds() methods defined by Component.

Window

- ✓ The Window class creates a top-level window.
- ✓ A window objects is a top-level window with no borders and no menu bar.
- ✓ It doesn't contain any components.
- ✓ It has two subclasses; Frame is window with name and menu bar and Dialog to represent dialog boxes.

AWT Controls

AWT Controls Labels

- ✓ The easiest control to use is a label.
- ✓ A *label* is an object of type **Label**, and it contains a string, which it displays.
- ✓ Labels are passive controls that do not support any interaction with the user.
- ✓ **Label** defines the following constructors:

  - ❖ Label( )
  - ❖ Label(String *str*)
  - ❖ Label(String *str*, int *how*)

- ✓ The first version creates a blank label.
- ✓ The second version creates a label that contains the string specified by *str*. This string is left- justified.
- ✓ The third version creates a label that contains the string specified by *str* using the alignment specified by *how*.
- ✓ The value of *how* must be one of these three constants:
  - ❑ **Label.LEFT**,
  - ❑ **Label.RIGHT**
  - ❑ **Label.CENTER**.

Labels - methods

- **setText( ) :** It is used to set or change the text in a label.

  void setText(String *str*)

- **getText():** It is used to obtain the current label.

  String getText( )

✓ For **setText( )**, *str* specifies the new label. For **getText( )**, the current label is returned.

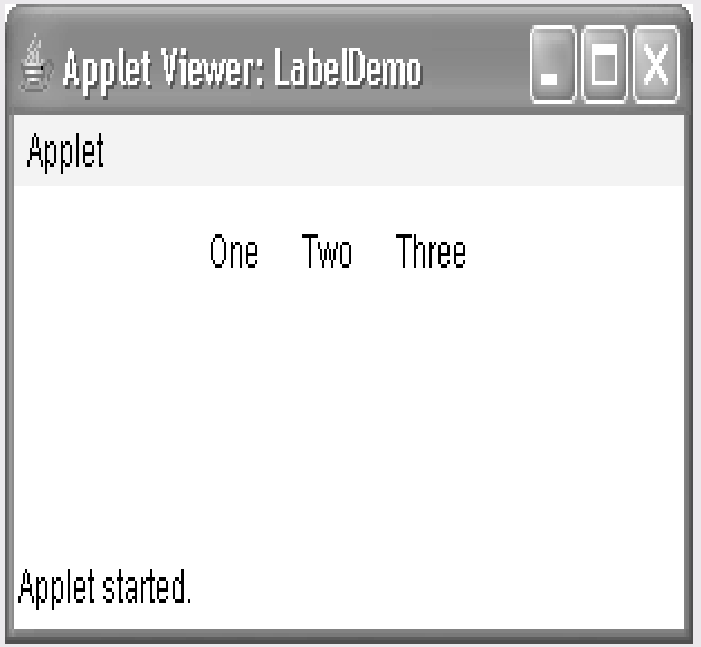
- **setAlignment( ) :**It is used to set the alignment of the string within the label.

  void setAlignment(int *how*)

- **getAlignment( ):**It is used To obtain the current alignment,

  int getAlignment( )

```
import java.awt.*;
import java.applet.*;
/*
<applet code="LabelDemo" width=300 height=200>
</applet>
*/
 public class LabelDemo extends Applet
{
     public void init()
     {
          Label one = new Label("One");
          Label two = new Label("Two");
          Label three = new Label("Three");
          // add labels to applet window
          add(one);
          add(two);
          add(three);
     }
}
```



Buttons

- ✓ *A push button* is a component that contains a label and that generates an event when it is pressed.
- ✓ Push buttons are objects of type **Button**.
- ✓ **Button** defines these two constructors:
  - ❑ Button( )
  - ❑ Button(String *str*)
- ✓ The first version creates an empty button.
- ✓ The second creates a button that contains *str* as a label.

setLabel( ):

- ✓ It is used to set label to button

  void setLabel(String *str*)

getLabel( ):

- ✓ It is used to get the label on the button

  String getLabel( )

Here, *str* becomes the new label for the button

Handling Buttons

- ✓ Each time a button is pressed, an action event is generated.
- ✓ This is sent to any **ActionListener** that previously registered to receive action event notifications from that component.
- ✓ **ActionListener** interface defines the **actionPerformed( )** method, which is called when an event occurs.
- ✓ An **ActionEvent** object is supplied as the argument to this method
- ✓ It contains both a reference to the button that generated the event and label of the button.

Check Boxes

- ✓ *Acheck box* is a control that is used to turn an option on or off.
- ✓ It consists of a small box that can either contain a check mark or not.
- ✓ There is a label associated with each check box that describes what option the box represents. You change the state of a check box by clicking on it.
- ✓ Check boxes can be used individually or as part of a group.
- ✓ Check boxes are objects of the **Checkbox** class.

Checkbox supports these constructors:

- Checkbox( )
- Checkbox(String *str*)
- Checkbox(String *str*, boolean *on*)
- Checkbox(String *str*, boolean *on*, CheckboxGroup *cbGroup*)
- Checkbox(String *str*, CheckboxGroup *cbGroup*, boolean *on*)

- ✓ The first form creates a check box whose label is initially blank. The state of the check box is unchecked.
- ✓ The second form creates a check box whose label is specified by *str*. The state of the check box is unchecked.
- ✓ The third form allows you to set the initial state of the check box. If *on* is **true**, the check box is initially checked; otherwise, it is cleared.
- ✓ The fourth and fifth forms create a check box whose label is specified by *str* and whose group is specified by *cbGroup*.

Methods:

- ✓ **getState( )** :To retrieve the current state of a check box.
- ✓ **setState( )** :To set its state.
- ✓ **getLabel( )** :we can obtain current label associated with a check box
- ✓ **setLabel( )** :To set the label

Syntax :

```
boolean getState( )
void setState(boolean on)
String getLabel( )
void setLabel(String str)
```

Handling Check Boxes :

- ✓ Each time a check box is selected or deselected, an item event is generated.
- ✓ This is sent to **ItemListene** that previously registered to receive an item event notifications from that component.
- ✓ **ItemListene** interface defines the **itemStateChanged( )** method.
- ✓ An **ItemEvent** object is supplied as the argument to this method. It contains information about the event

Checkbox Group : Radio buttons

- ✓ It is possible to create a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time.
- ✓ These check boxes are often called *radio buttons*.
- ✓ To create a set of mutually exclusive check boxes, we must first define the group to which they will belong and then specify that group when we construct the check boxes.
- ✓ Check box groups are objects of type CheckboxGroup.

```
CheckboxGroup cbg=new CheckboxGroup();
```

- **getSelectedCheckbox( )** : It determines which check box in a group is currently selected.
- **setSelectedCheckbox( )** : It sets a check box.

Syntax : **Checkbox getSelectedCheckbox( )**

void setSelectedCheckbox(Checkbox *which*)

here,
which → check box that we want to be selected.

List

- ✓ The **List** class is used to create a pop-up list.
- ✓ **Choice** object, which shows only the single selected item in the menu,
- ✓ **List** object can be constructed to show any number of choices in the visible window
- ✓ From list it is possible to select more than one item.
- ✓ It provides multiple-choice, scrolling selection list.
- ✓ It can also be created to allow multiple selections.
- ✓ **List** provides these constructors:
  - ❑ List( )
  - ❑ List(int *numRows*)
  - ❑ List(int *numRows*, boolean *multipleSelect*)

- The first version creates a **List** control that allows only one item to be selected at any one time.
- In the second form, the value of *numRows* specifies the number of entries in the list that will always be visible.
- In the third form, if *multipleSelect* is **true**, then the user may select two or more items at a time. If it is **false**, then only one item may be selected.

List- methods

add(): It is used to add items to the list

void add(String *name*)
void add(String *name*, int *index*)

- Here, *name* is the name of the item added to the list.
- The first form adds items to the end of the list.
- The second form adds the item at the index specified by *index*. Indexing begins at zero.

getSelectedItem( ): used to get the name of selected item.

```
String getSelectedItem( )
```

getSelectedIndex(): used to get the selected item index.

```
int getSelectedIndex( )
```

getSelectedItems( ) or getSelectedIndexes( ) :

```
String[ ] getSelectedItems( )
int[ ] getSelectedIndexes( )
```

getSelectedItems( ): returns an array containing the names of the currently selected items.

getSelectedIndexes( ): returns an array containing the indexes of the currently selected items.

getItemCount( ): It obtain the number of items in the list

Syntax: int getItemCount( )

getItem( ): Given an index, we can obtain the name associated with the item at that index

Syntax: String getItem(int *index*)

Handling Lists:

- ✓ Each time a item is selected in a list, an item event is generated. This is sent to **ItemListener** that previously registered to receive an item event notifications from that component.
- ✓ **ItemListener** interface defines the **itemStateChanged( )** method.
- ✓ An **ItemEvent** object is supplied as the argument to this method. It contains information about the event

TextFields

- ✓ The **TextField** class implements a single-line text-entry area, usually called an *edit control*.
- ✓ Text fields allow the user to enter strings and to edit the text using the arrow keys, cut and paste keys, and mouse selections.
- ✓ **TextField** is a subclass of **TextComponent**.
- ✓ **TextField** defines the following constructors:
  - TextField( )
  - TextField(int *numChars*)
  - TextField(String *str*)
  - TextField(String *str*, int *numChars*)

- ✓ The first version creates a default text field.
- ✓ The second form creates a text field that is *numChars* characters wide.
- ✓ The third form initializes the text field with the string contained in *str*.
- ✓ The fourth form initializes a text field and sets its width.

TextFields- Methods:

getText( ): To obtain the string currently contained in the text field

setText( ): To set the text.

String getText( )

void setText(String *str*)

here, *str* is the new string.

select( ) : The user can select a portion of the text in a text field. Also, we can select a portion of text under program control

getSelectedText( ) : our program can obtain the currently selected text

Syntax :

String getSelectedText( )

void select(int *startIndex*, int *endIndex*)

getSelectedText( ) returns the selected text.

The select( ) method selects the characters beginning at *startIndex* and ending at *endIndex*–1.

setEditable( ): we can control whether the contents of a text field may be modified by the user or not.

isEditable( ): we can determine editability

Syntax:

boolean isEditable( )

void setEditable(boolean *canEdit*)

isEditable( ) returns true if the text may be changed and false if not.

In **setEditable( ),** if *canEdit* is true, the text may be changed. If it is false, the text cannot be altered.

setEchoChar( ) :

- ✓ There may be times when we will want the user to enter text that is not displayed, such as a password. We can disable the echoing of the characters as they are typed.
- ✓ This method specifies a single character that the TextField will display when characters are entered (thus, the actual characters typed will not be shown and we can display like * or $ …..).

echoCharIsSet( ): we can check a text field to see if it is in echoChar mode.

getEchoChar( ): we can retrieve the echo character.

Syntax:

```
void setEchoChar(char ch)
boolean echoCharIsSet( )
char getEchoChar( )
```

here, *ch* specifies the character to be echoed.

TextArea

- ✓ Sometimes a single line of text input is not enough for a given task. To handle these situations, the AWT includes a simple multiline editor that is textarea.
- ✓ In text area you may enter more than one line of text.

Constructors :

TextArea( )
TextArea(int *numLines*, int *numChars*)
TextArea(String *str*)
TextArea(String *str*, int *numLines*, int *numChars*)
TextArea(String *str*, int *numLines*, int *numChars*, int *sBars*)

here, *numLines* → the height, in lines, of the text area. *numChars* → its width, in characters. *str* →Initial text.

It supports the getText( ), setText( ), getSelectedText( ), select( ), isEditable( ), and setEditable( ) methods.

Button (java.awt. Button)

To create a Button object, simply create an instance of the Button class by calling one of the constructors. The most commonly used constructor of the Button class takes a String argument that gives the Button object a text title. The two constructors are:

```
Button ()                 // Constructs a Button with no label.
Button (String label) // Constructs a Button with the specified label.
```

Figure: A Button Component

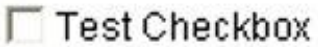
Checkboxes (java.awt. Checkbox)

Checkboxes have two states, on and off. The state of the button is returned as the Object argument, when a Checkbox event occurs. To find out the state of a checkbox object we can use getState () that returns a true or false value. We can also get the label of the checkbox using getLabel () that returns a String object.

```
Checkbox checkbox1 = new Checkbox ("Java");

Checkbox checkbox1 = new Checkbox ("C#");
```

Figure: A Checkbox Component

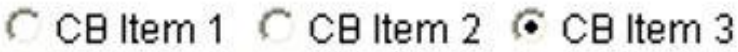


Radio Buttons (java.awt.CheckboxGroup)

Is a group of checkboxes, where only one of the items in the group can be selected at any one time.

```
CheckboxGroup cbg = new CheckboxGroup ();
Checkbox checkBox1 = new Checkbox ("C++", cbg, x);
```

Figure: A Radio Button Component



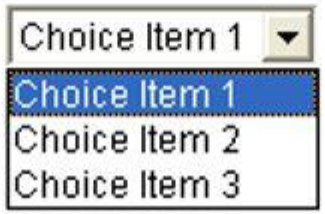
Choice Buttons (java.awt. Choice)

Like a radio button, where we make a selection, however it requires less space and allows us to add items to the menu dynamically using the addItem() method.

```
c = new Choice ();

c.add ("Java");
c.add ("C#");
c.add ("Python");
```

Figure: A Choice Button Component



Labels (java.awt.Label)

Allow us to add a text description to a point on the applet or application.

```
Label l1=new Label ("Test Label");
```

Figure: A Label Component

Test Label

TextFields (java.awt.TextField)

It is an area where the user can enter text. They are useful for displaying and receiving text messages.

```
TextField text1 = new TextField();
```

Figure: A TextField Component

Simple Registration Form


```
Label namelabel = new Label("Name:              ");
Label emaillabel = new Label("Email:             ");
Label passwordlabel = new Label("Password:      ");
Label confirmPasswordlabel = new Label("Confirm Password:");
Label genderlabel = new Label("Gender:");
Label countryLabel = new Label("Branch");
TextField name = new TextField(28);
TextField email= new TextField(28);
TextField password = new TextField(28);
TextField confirmPassword = new TextField(25);
```

```
CheckboxGroup genderGroup = new CheckboxGroup();
Checkbox maleCheckbox = new Checkbox("Male", genderGroup, false);
Checkbox femaleCheckbox = new Checkbox("Female
Choice countryChoice = new Choice();
countryChoice.add("Select");
countryChoice.add("India");
countryChoice.add("USA");
countryChoice.add("Paris");
countryChoice.add("China");

password.setEchoChar('*');
confirmPassword.setEchoChar('*');

Button submitButton = new Button("Submit");
```

```
// Add components to the applet
        add(namelabel);
        add(name);
        add(emaillabel);
        add(email);
        add(passwordlabel);
        add(password);
        add(confirmPasswordlabel);
        add(confirmPassword);
        add(genderlabel);
        add(maleCheckbox);
        add(femaleCheckbox);
        add(countryLabel);
        add(countryChoice);
        add(new Label("    "));
        add(submitButton);
```

Delegation Event Model

Event: The events are defined as an object that describes a change in the state of a source object. An event defines the change in the state of an event source.

User want the submit button to perform some action, when he clicks the button. Clicking a button, typing text in a text box, item selection in a list and so on.

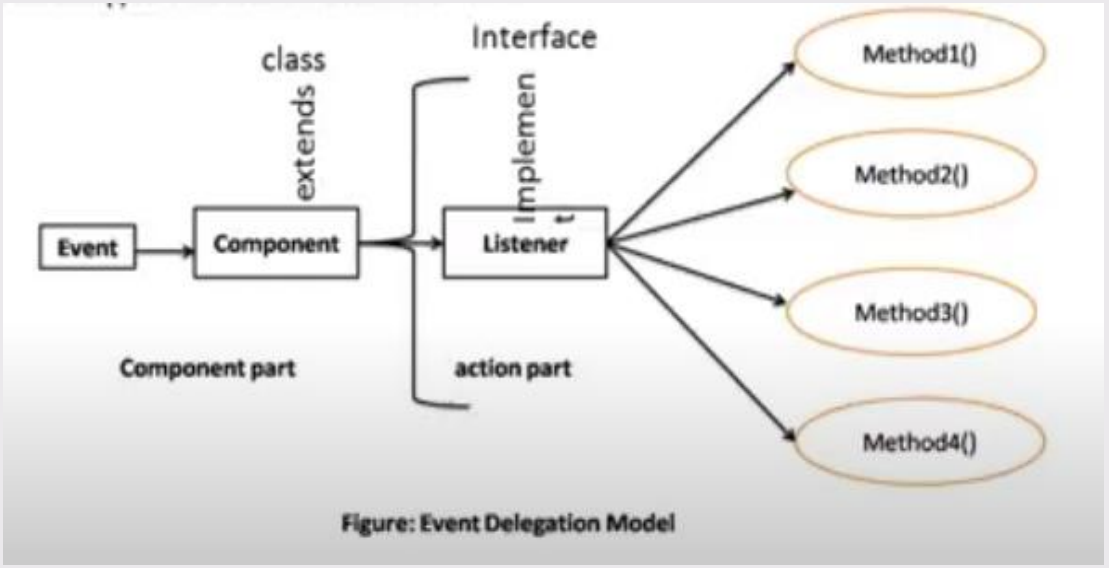


Figure: Event Delegation Model

The modern approach to handling events is based on the **delegation event model**, which defines standard and consistent mechanisms to generate and process events.

Events, Sources, and Listeners

The delegation event model can be defined by three components: event, event source, and event listeners.

Events: The event object defines the change in state in the event source class. For example, interacting with the graphical interfaces, such as clicking a button or entering text via keyboard in a text box, item selection in a list, all represent some sort of change in the state.

Event sources: Event sources are objects that cause the events to occur due to some change in the property of the component. An Event source is an object that generates an event. This occurs when the internal state of that object changes in some way.

Event listeners: Event listeners are objects that are notified as soon as a specific event occurs. Event listeners must define the methods to process the notification they are interested to receive. It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications.

Types of Event classes

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Java Event classes and Listener interfaces

Event Classes	Listener Interfaces
ActionEvent	ActionListener
MouseEvent	MouseListener and MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

write a Java applet code that displays the text 'Java Programming' when a button is clicked.

```
public class Message extends Applet
implements ActionListener
{
    String message = "";
    Button b;

    public void init() {
      b = new Button("Click Me");
      b.addActionListener(this);
      add(b);
    }
     public void actionPerformed(ActionEvent e)
    {
      if (e.getSource() == b) {
          message = "Java Programming";
          repaint();
      }
    }
     public void paint(Graphics g) {
      g.drawString(message, 50, 50);
    }
}
```

Java programs are generally classified into two ways

- **Applications programs**
- **Applet programs.**

Application Programs:

Applications programs are those programs normally created, compiled and executed as similar to the other languages. Each Application program contains one main() method. Programs are created in a local computer. Programs are compiled with javac compiler and executed with java interpreter.

Applet Programs:

Applet programs are small programs that are primarily used in Internet programming. These programs are either developed in local systems or in remote systems and are executed by either a java compatible "Web browser" or "appletviewer".

- An applet developed locally and stored in a local system is known as a ***Local Applet***. When a Web page is trying to find a local applet, it does not need to use the Internet and therefore the local system does not require the Internet connection.

- An applet developed by someone else and stored on a remote computer is known as ***Remote Applet***. If our system is connected to the Internet, we download the remote applet onto our system via the Internet and run it.

Applet class

- Applet class provides all necessary support for applet execution, such as initializing and destroying of applet.
- Applet provides several methods which are used to supports applet window-based activities.
- Some methods are shown below:

  - **public void init()**

    init() is the first method to be called. It is used to initialize the Applet.

  - **public void start()**

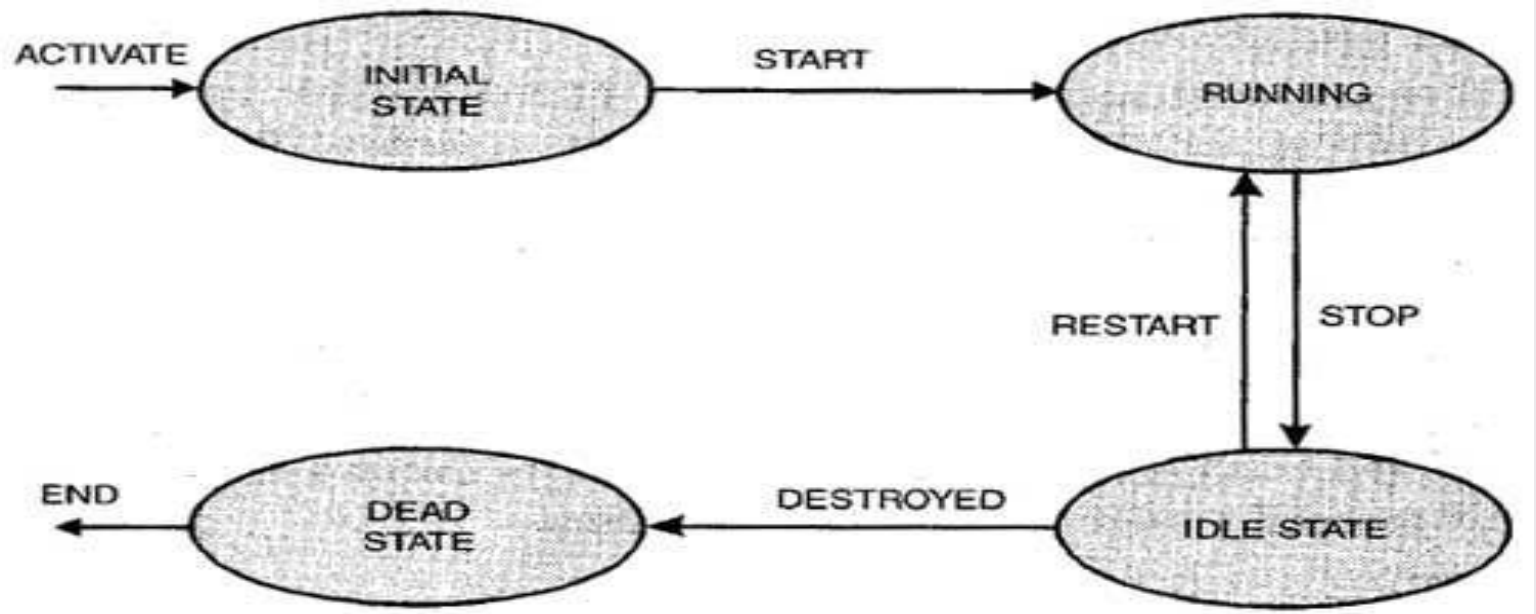
    It is automatically called after the init() method or browser is maximized.

- **public void stop()**
  It is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
- **public void paint()**
  It is used to paint the Applet. It is Invoked immediately after the start() method.
- **public void destroy()**
  destroy() method is called when your applet needs to be removed completely from memory.
- **boolean isActive( )**
  Returns true if the applet has been started. It returns false if the applet has been stopped.
- **void showStatus(String str)**
  Displays str in the status window of the browser or applet viewer.

Applet life cycle

Life Cycle of an Applet(Applet skeleton)

- Initialization State
- Running State
- Idle or Stopped State
- Dead State



Life Cycle of an Applet

Initialization State:

- Applet enters the initialization state when it is first loaded.
- This is achieved by calling the **init()** method of **Applet** class.
- The applet is born.
- The initialization occurs only once in the applet's life cycle.
- Generally, all the initialization variables are to be placed in the init() method.

Syntax:

```
public void init()
{
    . . . .
    . . . .    (Action)
}
```

Running State:

- Applet enters the running state when the system calls the **start()** method of **Applet** class.
- This occurs automatically after the applet is initialized.
- Starting can also occur if the applet is already in "Stopped State".
- The start() method may be called more than once.

Syntax:

```
public void start()
{
    . . . .
    . . . .    (Action)
}
```

Idle or Stopped State:

- An applet becomes idle when it is stopped from running.
- Stopping occurs automatically when we leave the page containing the currently running applet.
- This can also be done by calling the **stop()** method explicitly.

Syntax:

```
public void stop()
{
    . . . .
    . . . .    (Action)
}
```

Dead State:

- An applet is said to be dead when it is removed from memory.
- This occurs automatically by invoking the **destroy()** method when we quit the browser.
- Destroying stage occurs only once in the applet life cycle.

Syntax:

```
public void destroy()
{
    . . . .
    . . . .   (Action)
}
```

Display State:

- Display state is useful to display the information on the output screen.
- This happens immediately after the applet enters into the running state.
- The **paint()** is called to accomplish this task.
- Almost every applet will have a paint() method.

Syntax:

```
public void paint(Graphics g)
{
    . . . .
    . . . .   (Display Statements)
}
```

Example:

```
import java.awt.*;
import java.applet.*;

/*
<applet code="AppletSkel" width=300 height=100>
</applet>
*/

public class AppletSkel extends Applet
{
public void init()
{
// initialization called first
}
```

```
public void start()
{
// start or resume execution
}

public void stop()
{
// suspends execution
}

public void destroy()
 {
// perform shutdown activities
}
```

```
public void paint(Graphics g)
{
// redisplay contents of window
}
}
```

Compilation: javac AppletSkel.java
Execution: appletviewer AppletSkel.java

Limitations of AWT

Limitations of AWT

The Abstract Window Toolkit (AWT) is Java's original platform-dependent GUI (Graphical User Interface) framework. Although it provides basic components like buttons, text fields, and menus, it has several limitations that led to the introduction of Swing in Java.

- Platform Dependence & Inconsistency
- Heavyweight Components
- Limited Component Library
- Poor Customization & Flexibility
- Technical Constraints

1. Platform Dependence & Inconsistency

AWT uses native peers, that is each UI component is created using the system's OS.

- **Inconsistent Look:** A button might appear differently or have different default dimensions depending on the OS.
- **Inconsistent Behavior:** Events and layouts may work differently across systems, so "Write Once, Run Anywhere" is not fully achieved.

2. Heavyweight Components

AWT components depend on the operating system, so they are called **heavyweight**.

- **Resource Consumption:** They use more memory and processing power than lightweight components in Swing.
- **Performance Overhead:** Using native code can make them slower and may cause synchronization issues.

3. Limited Component Library

AWT provides only a basic set of UI controls.

- **Missing Advanced Features:** It lacks essential modern components such as tables (JTable), trees (JTree), sliders, and tabbed panes.
- **No Image Support for Buttons:** Standard AWT buttons cannot display images or icons.

4. Poor Customization & Flexibility

AWT offers very little control over the visual style of components.

- **Fixed Look and Feel:** Developers cannot easily change colors, borders, or styles because the OS handles the rendering.
- **Lack of Pluggable Look and Feel:** Unlike Swing, you cannot change the theme of an AWT application at runtime.

5. Technical Constraints

- **Integer Coordinate Limits:** Component sizes and locations are stored as integers, which may be further restricted by specific platform limits.
- **No MVC Support:** AWT does not follow the Model-View-Controller (MVC) architecture, which makes code organization for complex applications more difficult.
- **Basic Graphics:** The API lacks advanced features like anti-aliasing, transparency, and 3D graphics support.

Exploring Swing

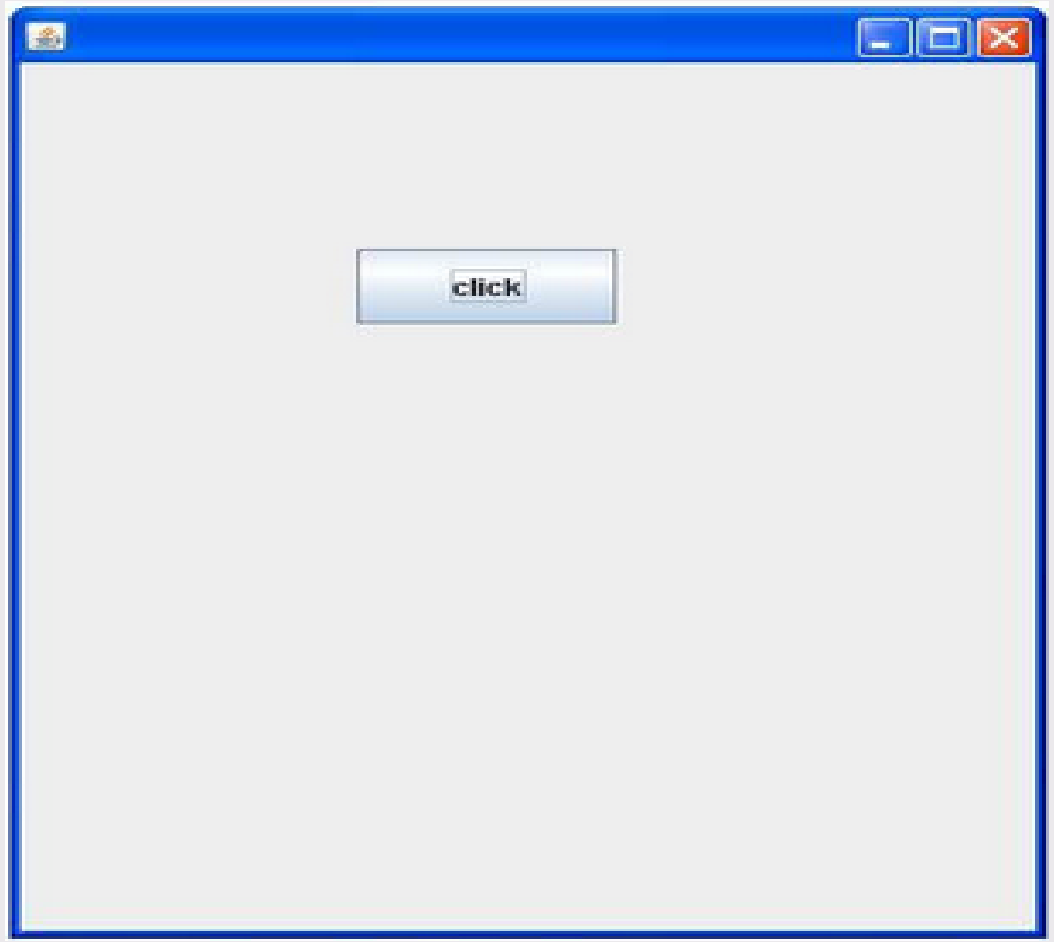
Exploring Swing

- It is part of Oracle's Java Foundation Classes(JFC) – an API for providing a graphical user interface (GUI) for Java programs.
- Swing was developed to provide a more sophisticated set of GUI components than the earlier Abstract Window Toolkit (AWT)
- Swing and packaged in **javax.swing**
- The Swing component classes

JButton	JCheckBox	JComboBox	JLabel
JList	JRadioButton	JScrollPane	JTabbedPane
JTable	JTextField	JToggleButton	JTree

Simple Java Swing Example

```
import javax.swing.*;
public class FirstSwingExample extends Swing {
public static void main(String[] args) {
JFrame f=new JFrame();
JButton b=new JButton("click");
b.setBounds(130,100,100, 40);
 f.add(b);
 f.setSize(400,500);
f.setLayout(null);
f.setVisible(true);
}
}
```



JLabel and ImageIcon

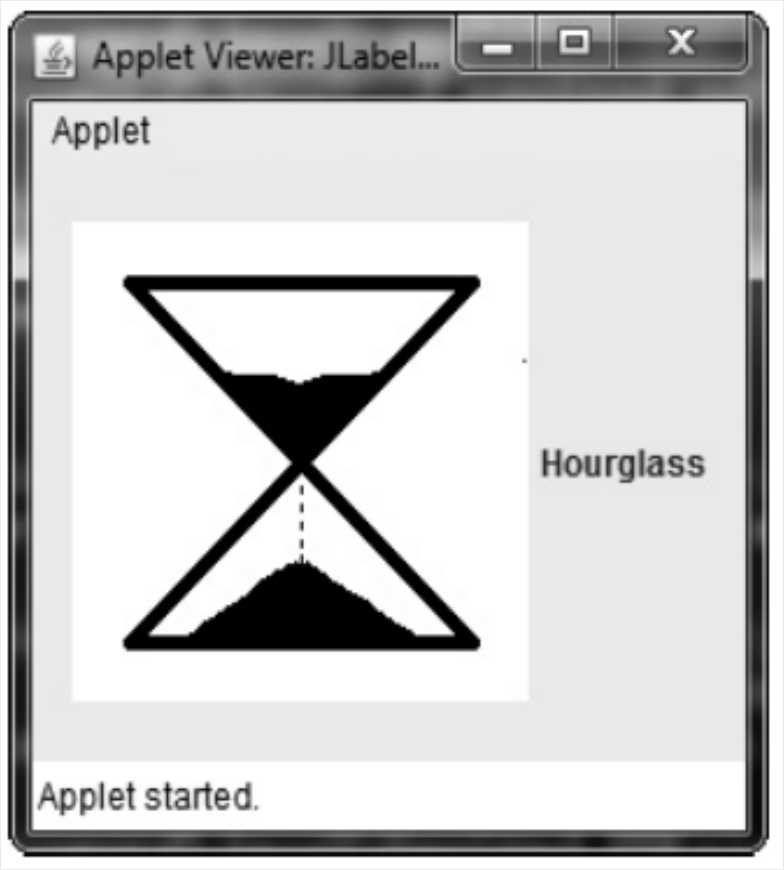
- ✓ **JLabel** is Swing's easiest-to-use component.
- ✓ **JLabel** defines several constructors.
  - ➢ JLabel(Icon *icon*)
  - ➢ JLabel(String *str*)
  - ➢ JLabel(String *str*, Icon *icon*, int *align*)
- ✓ Notice that icons are specified by objects of type **Icon**, which is an interface defined by Swing.
- ✓ The easiest way to obtain an icon is to use the **ImageIcon** class.
- ✓ **ImageIcon** implements **Icon** and encapsulates an image.
- ✓ Thus, an object of type **ImageIcon** can be passed as an argument to the **Icon** parameter of **JLabel**'s constructor.

JLabel and ImageIcon

Methods	Description
String getText()	It returns the text string that a label displays.
void setText(String text)	It defines the single line of text this component will display.
void setHorizontalAlignment (int alignment)	It sets the alignment of the label's contents along the X axis.
Icon getIcon()	It returns the graphic image that the label displays.
int getHorizontalAlignment()	It returns the alignment of the label's contents along the X axis.

JLabel and ImageIcon EXample


```
import java.awt.*;
import javax.swing.*;
public class LabelExample {
    public static void main(String[] args) {
      JFrame f = new JFrame("Label Example");
      ImageIcon ii = new ImageIcon("hourglass.png");
      JLabel jl = new JLabel("Hourglass", ii, JLabel.CENTER);
      jl.setBounds(50, 50, 200, 100);
      f.add(jl);
      f.setSize(300, 300);
      f.setLayout(null);
      f.setVisible(true);
      f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```



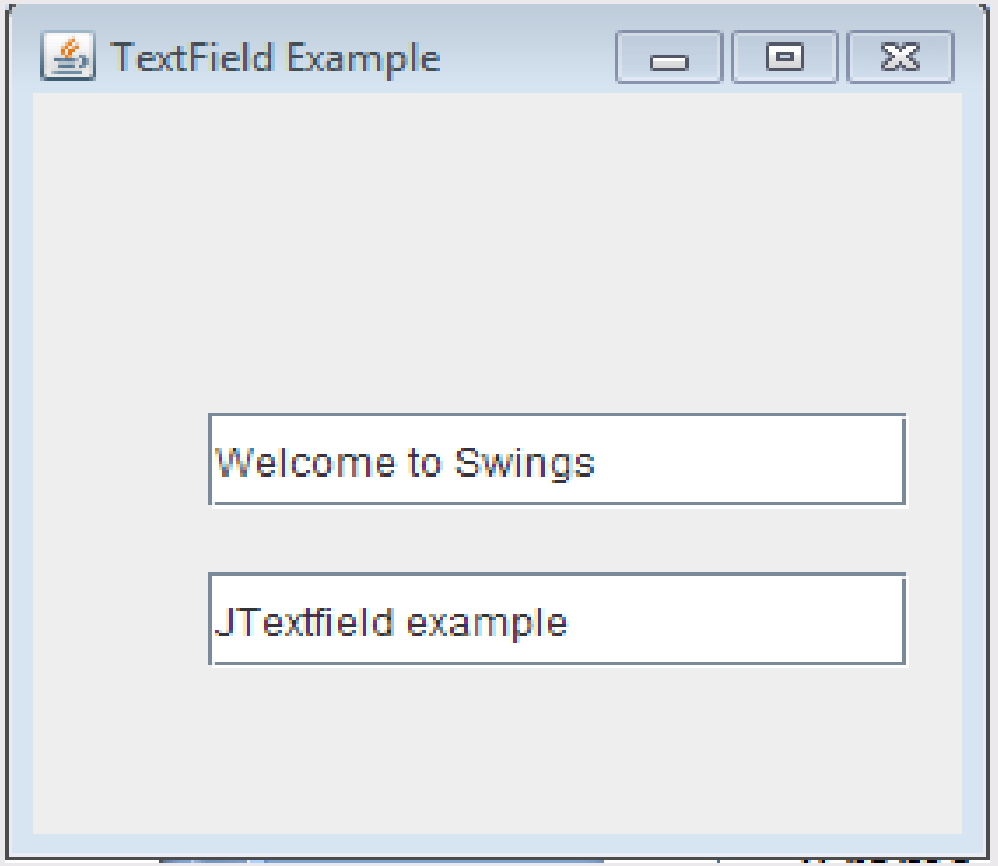
JTextField

- ✓ The object of a JTextArea class is a multi line region that displays text.
- ✓ It allows the editing of multiple line text. It inherits JTextComponent class.
- ✓ public class JTextArea extends JTextComponent

Methods	Description
void setRows(int rows)	It is used to set specified number of rows.
void setColumns(int cols)	It is used to set specified number of columns.
void setFont(Font f)	It is used to set the specified font.
void insert(String s, int position)	It is used to insert the specified text on the specified position.
void append(String s)	It is used to append the given text to the end of the document.

JTextField Example

```
import javax.swing.*;
class TextFieldExample  {
      public static void main(String args[]) {
            JFrame f= new JFrame("TextField Example");
            JTextField t1,t2;
            t1=new JTextField("Welcome to Swings");
            t1.setBounds(50,100, 200,30);
            t2=new JTextField("JTextfield example");
            t2.setBounds(50,150, 200,30);
            f.add(t1); f.add(t2);
            f.setSize(400,400);
            f.setLayout(null);
            f.setVisible(true);
    }
}
```



JButton

- ✓ The JButton class is used to create a labeled button that has platform independent implementation.
- ✓ The application result in some action when the button is pushed. It inherits AbstractButton class.
- ✓ The declaration for javax.swing.JButton class.
- ✓ public class JButton extends AbstractButton implements Accessible
- ✓ Methods of AbstractButton class

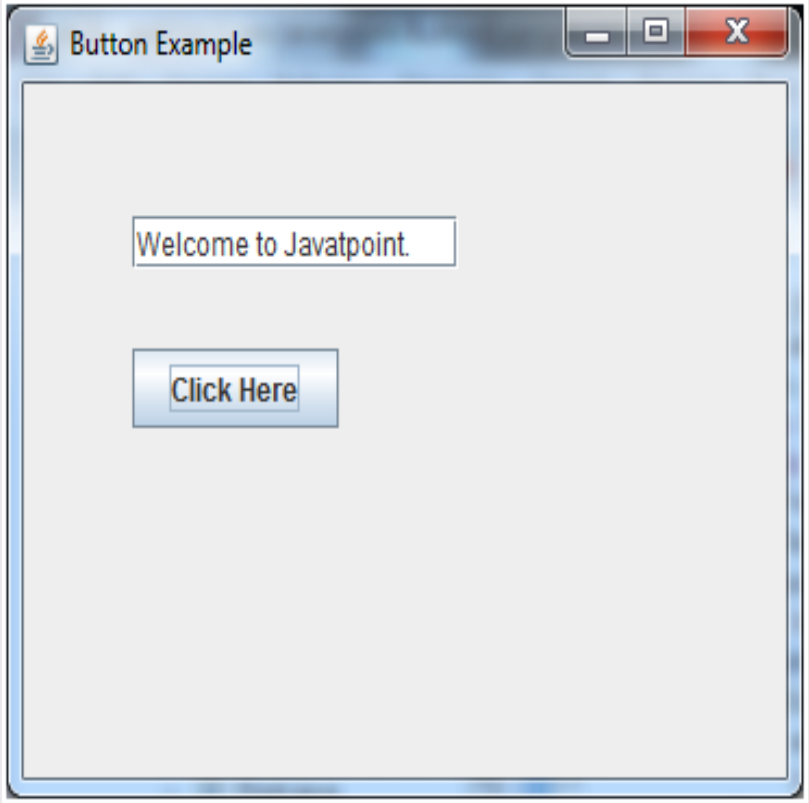
JButton

Methods	Description
void setText(String s)	It is used to set specified text on button
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener (ActionListener a)	It is used to add the action listener to this object.

Jbutton Example

```
import java.awt.event.*;
import javax.swing.*;
public class ButtonExample {
    public static void main(String[] args) {
        JFrame f = new JFrame("Button Example");
        final JTextField tf = new JTextField();
        tf.setBounds(50, 50, 150, 20);
        JButton b = new JButton("Click Here");
        b.setBounds(50, 100, 95, 30);
        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                tf.setText("Welcome to Javatpoint.");
            }
        });
```

```
        f.add(b);
        f.add(tf);
        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```



JCheckBox

- ✓ The JCheckBox class is used to create a checkbox. It is used to turn an option on (true) or off (false).
- ✓ Clicking on a CheckBox changes its state from "on" to "off" or from "off" to "on ".
- ✓ It inherits JToggleButton class.
- ✓ public class JCheckBox extends JToggleButton implements Accessible

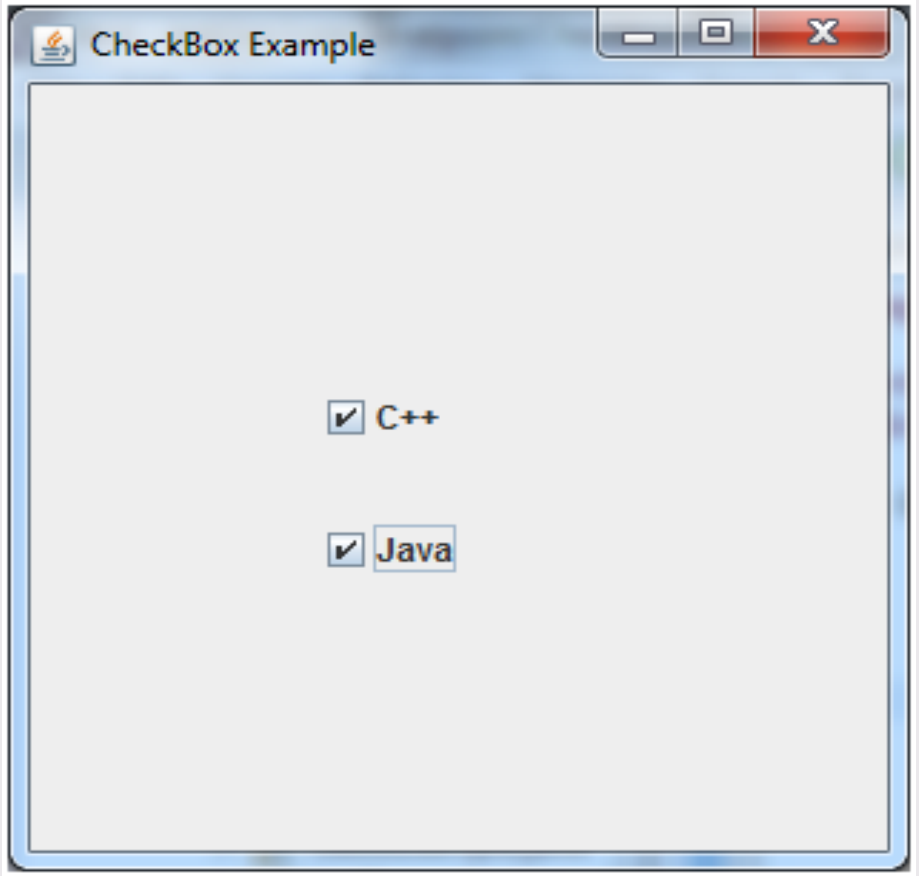
Methods	Description
AccessibleContext getAccessibleContext()	It is used to get the AccessibleContext associated with this JCheckBox.
protected String paramString()	It returns a string representation of this JCheckBox.

JCheckBox Example

```
import javax.swing.*;

public class CheckBoxExample {
    CheckBoxExample() {
        JFrame f = new JFrame("CheckBox Example");
        JCheckBox checkBox1 = new JCheckBox("C++");
        checkBox1.setBounds(100, 100, 100, 50);
        JCheckBox checkBox2 = new JCheckBox("Java", true);
        checkBox2.setBounds(100, 150, 100, 50);
        f.add(checkBox1);
        f.add(checkBox2);
        f.setSize(400, 400);
```

```
        f.setLayout(null);
       f.setVisible(true);
       f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String args[]) {
        new CheckBoxExample();
    }
}
```



JRadioButton

- ✓ The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options.
- ✓ It is widely used in exam systems or quiz.
- ✓ It should be added in ButtonGroup to select one radio button only.
- ✓ public class JRadioButton extends JToggleButton implements Accessible

JRadioButton

Methods	Description
void setText(String s)	It is used to set specified text on button.
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener (ActionListener a)	It is used to add the action listener to this object.

JRadioButton Example

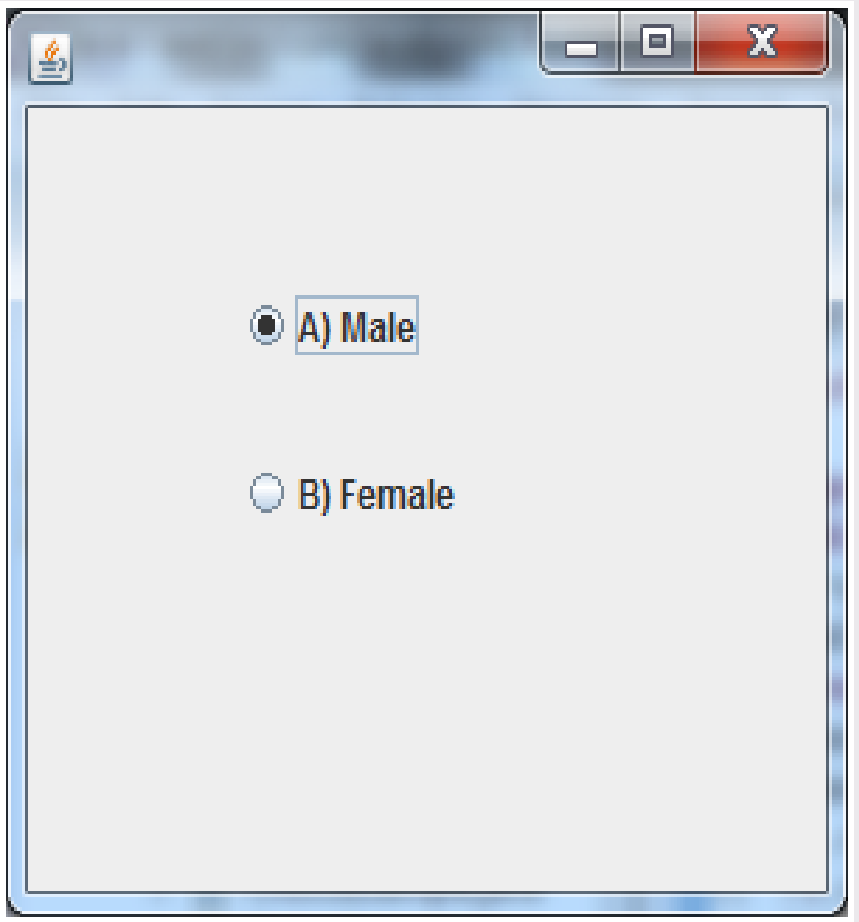

```
import javax.swing.*;

public class RadioButtonExample {
    JFrame f;

    RadioButtonExample() {
        f = new JFrame("Radio Button Example");
        JRadioButton r1 = new JRadioButton("A) Male");
        JRadioButton r2 = new JRadioButton("B) Female");
        r1.setBounds(75, 50, 100, 30);
        r2.setBounds(75, 100, 100, 30);
        ButtonGroup bg = new ButtonGroup();
```

```
        bg.add(r1);
        bg.add(r2);
        f.add(r1);
        f.add(r2);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        new RadioButtonExample();
    }
}
```



JTabbedPane

- ✓ The JTabbedPane class is used to switch between a group of components by clicking on a tab with a given title or icon.
- ✓ It inherits JComponent class.
- ✓ public class JTabbedPane extends JComponent implements Serializable, Accessible,SwingConstants.

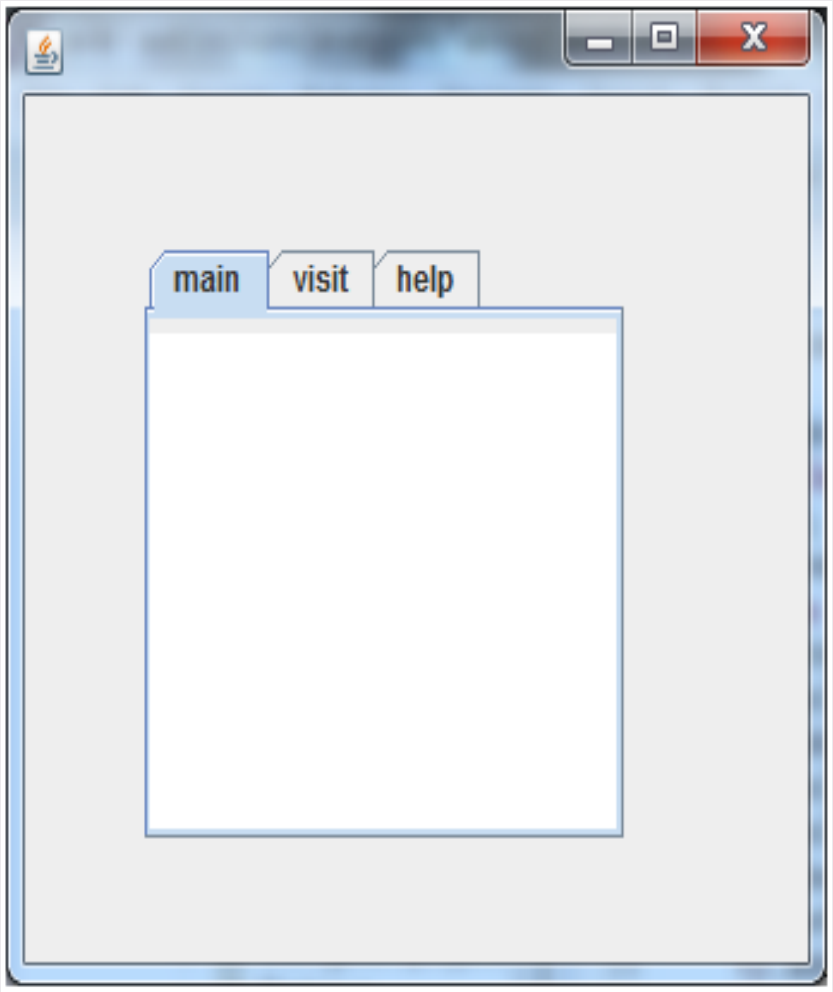
JTabbedPane Example

```
import javax.swing.*;

public class TabbedPaneExample {
    JFrame f;
    TabbedPaneExample() {
        f = new JFrame("TabbedPane Example");
        JTextArea ta = new JTextArea(10, 30);
        p1 = new JPanel();
        p1.add(ta);
        JPanel p2 = new JPanel();
        JPanel p3 = new JPanel();
        JTabbedPane tp = new JTabbedPane();
```

```
        tp.setBounds(50, 50, 200, 200);
        tp.add("Main", p1);
        tp.add("Visit", p2);
        tp.add("Help", p3);
        f.add(tp);
        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        new TabbedPaneExample();
    }
}
```



JList

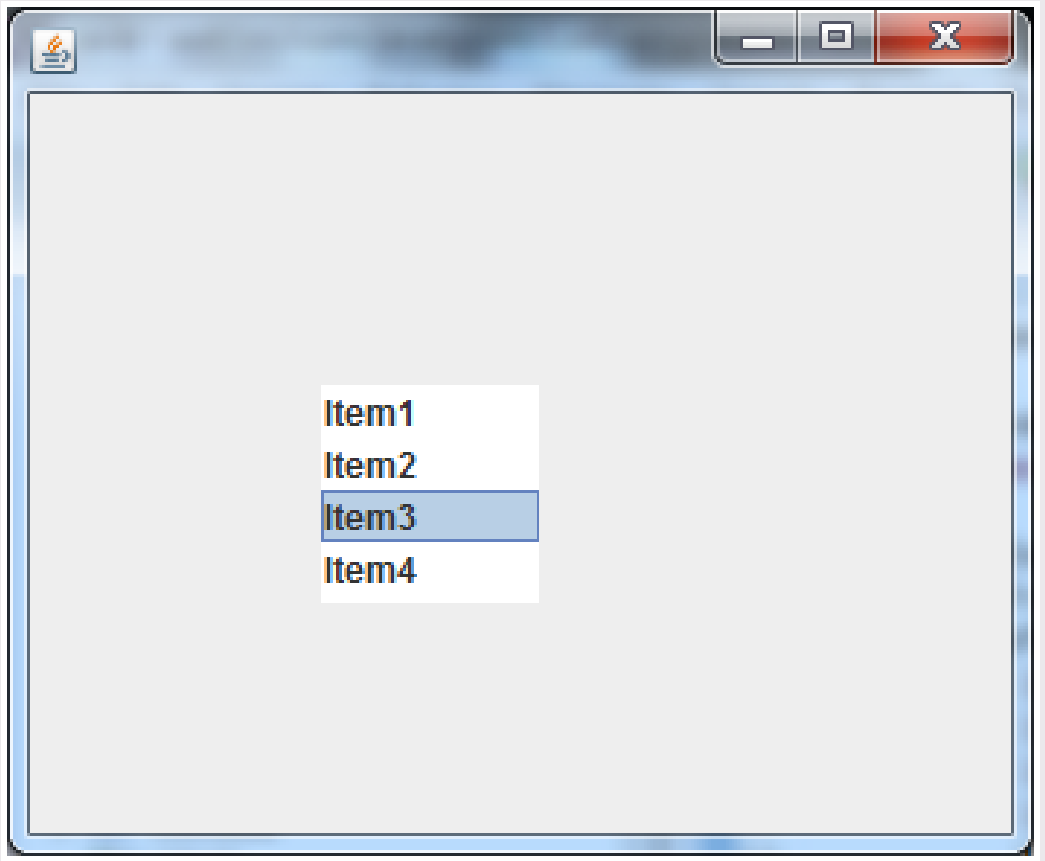
- ✓ The object of JList class represents a list of text items. The list of text items can be set up so that the user can choose either one item or multiple items. It inherits JComponent class.
- ✓ public class JList extends JComponent implements Scrollable, Accessible
- ✓ Commonly used Methods:

Methods	Description
Void addListSelectionListener (ListSelectionListener listener)	It is used to add a listener to the list, to be notified each time a change to the selection occurs.
int getSelectedIndex()	It is used to return the smallest selected cell index.

Methods	Description
ListModel getModel()	It is used to return the data model that holds a list of items displayed by the JList component.
void setListData(Object[] listData)	It is used to create a read-only ListModel from an array of objects.

```
import javax.swing.*;
public class ListExample
{
 ListExample(){
      JFrame f= new JFrame();
      DefaultListModel<String> l1 = new DefaultListModel<>();
```

```
        l1.addElement("Item1");
        l1.addElement("Item2");
        l1.addElement("Item3");
        l1.addElement("Item4");
        JList<String> list = new JList<>(l1);
        list.setBounds(100,100, 75,75);
        f.add(list);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
public static void main(String args[])
{
        new ListExample();
    }
```



JComboBox

- ✓ The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu.
- ✓ It inherits JComponent class.
- ✓ public class JComboBox extends JComponent implements ItemSelectable, ListDataListener, ActionListener, Accessible

Methods	Description
void addItem(Object anObject)	It is used to add an item to the item list.
void removeItem(Object anObject)	It is used to delete an item to the item list.
void removeAllItems()	It is used to remove all the items from the list.

void setEditable(boolean b)	It is used to determine whether the JComboBox is editable.
void addActionListener(ActionListener a)	It is used to add the ActionListener.
void addItemListener(ItemListener i)	It is used to add the ItemListener

```
import javax.swing.*;
public class ComboBoxExample
{
    JFrame f;
    ComboBoxExample(){
    f=new JFrame("ComboBox Example");
    String country[]={"India","Aus","U.S.A","England","Newzealand"};
```

```
    JComboBox cb=new JComboBox(country);
    cb.setBounds(50, 50,90,20);
    f.add(cb);
    f.setLayout(null);
    f.setSize(400,500);
    f.setVisible(true);
    }
   public static void main(String[] args)
   {
    new ComboBoxExample();
    }
}
```

Summary

- ✓ Swing was developed to provide a more sophisticated set of GUI components than the earlier Abstract Window Toolkit (AWT)
- ✓ Unlike AWT, Java Swing provides platform-independent and lightweight components.
- ✓ Swing Components are:

JButton	JCheckBox	JComboBox	JLabel
JList	JRadioButton	JScrollPane	JTabbedPane
JTable	JTextField	JToggleButton	JTree